

Chiffrement homomorphe basé sur LWE

Projet de Cryptographie (8INF874) — Implémentation logicielle et analyse
d'impact de la gestion de l'erreur



Groupe 6

Justin Bossard — BOSJ08070300

Samuel Plet — PLES07110100

2026-06-22

Université du Québec à Chicoutimi (UQAC)

Introduction et Problématique

- **Le chaînon manquant de la confidentialité** : Les données sont protégées au repos et en transit, mais exposées en clair **en cours d'utilisation**.
- **Modèle de menace cloud** : Serveur **Honest-but-curious**. Nécessité d'opérer sur les données à l'aveugle.
- **Menace quantique** : Vulnérabilité des systèmes asymétriques classiques (RSA, ECC).
- **Cryptographie Post-Quantique (PQC)** : Standardisation d'algorithmes basés sur les problèmes de réseaux euclidiens.

- **Définition formelle** : Un schéma de chiffrement E est homomorphe si, pour une opération $\star$ dans le domaine chiffré et $\circ$ dans le domaine clair, l'égalité suivante est vérifiée :

$$E(m_1) \star E(m_2) = E(m_1 \circ m_2)$$

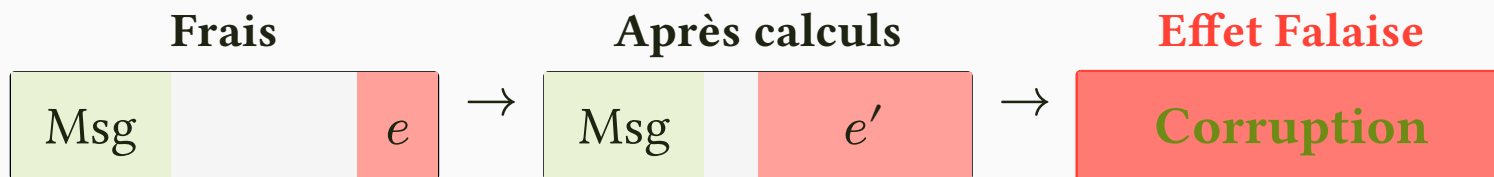
- **Propriétés** : Préserve la structure algébrique pour l'addition ($\oplus$) et la multiplication ($\otimes$).
- **Catégories** : PHE (Partiel, ex: Paillier) $\rightarrow$ SWHE (Limité) $\rightarrow$ FHE (Profondeur infinie).

- Problème calculatoire opérant sur des polynômes dans des anneaux quotients $\frac{\mathbb{Z}_q[X]}{X^n+1}$.
- **Génération des clés :**

$$b(x) = -a(x) \cdot s(x) + e(x) \pmod{q}$$

- ▶ $s(x)$: Polynôme secret (coefficients petits).
- ▶ $a(x)$: Polynôme aléatoire uniformément réparti.
- ▶ $e(x)$: Bruit (erreur probabiliste indispensable à la sécurité sémantique).
- **Rôle de l'erreur :** Sans le terme $e(x)$, le secret $s(x)$ est trivialement calculable.

- **Le paradoxe LWE** : Le bruit garantit la sécurité (IND-CPA), mais dégrade l'exactitude calculatoire.
- **Condition d'échec (Cliff Effect)** : Échec de déchiffrement si l'erreur accumulée e dépasse le facteur d'échelle Δ , empiétant sur les bits du message.



Architecture et Implémentation

- **BFV (Brakerski-Fan-Vercauteren) :**
 - Arithmétique entière **exacte**.
 - Le bruit cryptographique (LWE) est strictement isolé du message clair jusqu'à saturation.
- **CKKS (Cheon-Kim-Kim-Song) :**
 - Arithmétique à virgule flottante **approximative**.
 - Le bruit cryptographique se confond volontairement avec l'erreur de précision numérique.
- **Justification du choix (BFV) :** L'objectif de la preuve de concept est d'isoler et d'observer empiriquement l'**Effet Falaise** (épuisement du budget de bruit LWE). L'exactitude de BFV garantit une intégrité parfaite (erreur = 0) tant que le budget est positif, permettant une observation binaire de la corruption, impossible avec l'approximation inhérente à CKKS.

- **Multiplication de deux chiffrés** : Produit de $\left(c_0^{(1)}, c_1^{(1)}\right)$ et $\left(c_0^{(2)}, c_1^{(2)}\right)$.
- Génère intrinsèquement un **triplet** polynomial :

$$\left(c_0^{(1)}c_0^{(2)}, \quad c_0^{(1)}c_1^{(2)} + c_1^{(1)}c_0^{(2)}, \quad c_1^{(1)}c_1^{(2)}\right)$$

- **Relinéarisation** : Étape critique utilisant des clés d'évaluation publiques pour projeter le terme quadratique $(c_2 \cdot s^2)$ et ramener le texte chiffré à un format standard $(c_{0'}, c_{1'})$.
Opération source de la latence principale.

- **Degré du polynôme ($N = 8192$)** : Détermine le niveau de sécurité et la capacité d'encodage (Batching/SIMD).
- **Plain Modulus ($t = 2^{45}$)** :
 - Choisi pour éviter les dépassements arithmétiques (**overflows**) lors de l'évaluation du filtre Gamma.
 - Préserve l'espace du message face à la croissance exponentielle des coefficients.

- **Modèle asymétrique séparé** : Conception divisée entre orchestration (client simulé) et exécution arithmétique (serveur aveugle).

Frontend & API (Python/Flask)

- Serveur web interactif.
- Orchestration via subprocess.
- Rendu DOM et Métriques (JSON).
- Préparation image (Pillow).

⇒

Moteur FHE (C++17)

- Core FHE : Microsoft SEAL.
- Profilage CPU : `std::chrono`.
- I/O Images : `stb_image`.
- Build system : CMake.

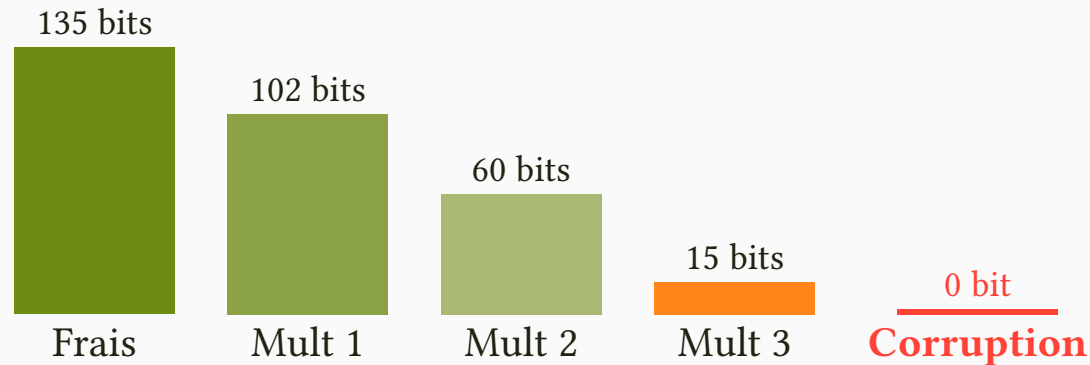
1. **Initialisation** : KeyGenerator instancie les clés de session (Secret, Public, RelinKeys).
2. **Encodage / Chiffrement** : BatchEncoder vectorise l'image (6400 pixels) dans les coefficients du polynôme, Encryptor génère le texte chiffré.
3. **Évaluation (Serveur)** : Evaluator applique multiply_inplace (correction Gamma) et add_plain_inplace (luminosité). La relinéarisation est forcée après chaque multiplication.
4. **Extraction des métriques** : Mesure empirique du bruit résiduel (invariant_noise_budget) et sérialisation des latences au format JSON.

Démonstration et Analyse

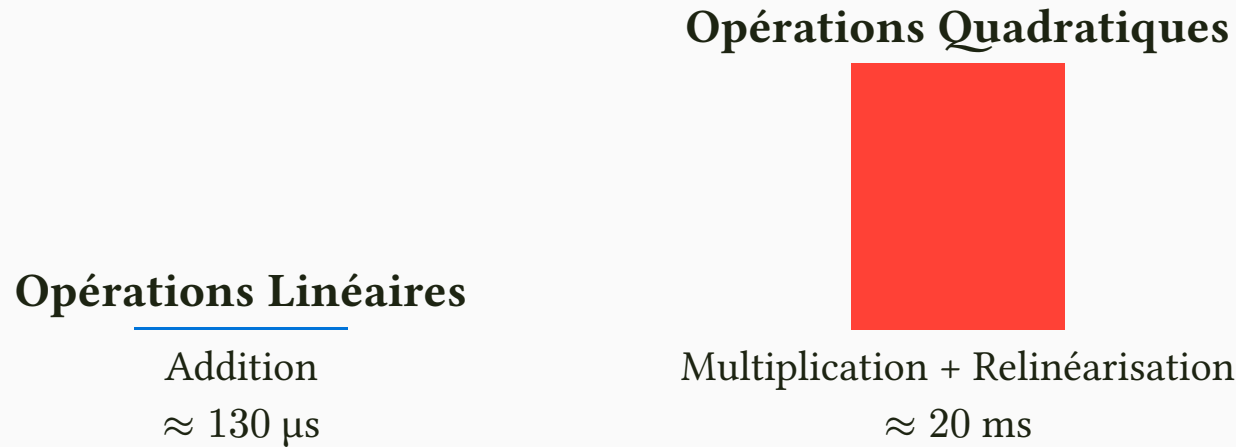
- **Contrainte** : L'opération polynomiale est coûteuse. Chiffrer un pixel par polynôme est inefficace.
- **Solution (Batching)** : Exploitation du théorème des restes chinois pour encoder plusieurs valeurs dans les coefficients d'un unique polynôme.
- **Application au POC** : Une image 80×80 (6400 pixels) tient dans un seul polynôme de degré $N = 8192$.



- Observation empirique de l'Effet Falaise lors de multiplications successives.
- Paramètres : $N = 8192$, Modulus = 45 bits. Budget initial ≈ 135 bits.



- Évaluation des performances sur l'opération unique via `std::chrono` (C++).
- L'asymétrie computationnelle justifie la nécessité d'optimisation matérielle.



- **Conclusion du POC :** La latence de traitement est acceptable pour des circuits fixes (Levelled-FHE). L'expansion mémoire nominale ($\frac{\text{Chiffré} \approx 100 \text{ Ko}}{\text{Clair } 8 \text{ octets}} \approx 10^4$) est partiellement amortie par l'encodage SIMD, mais reste le principal goulot d'étranglement réseau.

Conclusion et Perspectives

- Viabilité théorique des calculs sur données chiffrées démontrée.
- Garanties de sécurité post-quantiques validées par le schéma LWE.
- **Obstacles pratiques** : La gestion du budget de bruit et l'expansion mémoire limitent actuellement les applications en temps réel.

- **Définition** : Évaluation homomorphe du circuit de déchiffrement pour “nettoyer” le bruit sans révéler le clair.
- **État du calcul** :

FHE Niveau

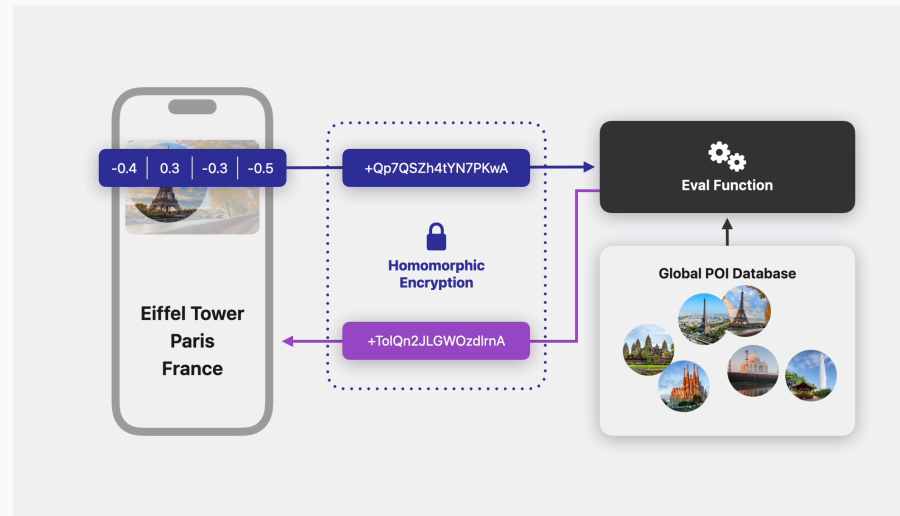
Capacité

Levelled-FHE $\longrightarrow$ Profondeur finie

Bootstrapping $\longrightarrow$ Profondeur infinie

- **Exclusion du POC** : Coût computationnel prohibitif, masquant l’observation de la croissance du bruit (objectif initial).

- Application du schéma BFV à la recherche visuelle sur appareil sans compromettre la confidentialité.



- **Validation de l'architecture asymétrique :**
 - **Client :** Extraction du vecteur d'embedding, quantification et chiffrement BFV.
 - **Serveur :** Évaluation homomorphe (produits scalaires) contre une base de données globale (POI).

- **Accélération matérielle (NTT) :**
 - La multiplication polynomiale est le goulot d'étranglement majeur. La NTT (**Number Theoretic Transform**) réduit sa complexité de $O(N^2)$ à $O(N \log N)$.
 - Les processeurs (CPU) sont limités par la bande passante mémoire. Les architectures dédiées (FPGA/ASIC) permettent une parallélisation massive pour viabiliser le FHE en temps réel.
- **Schémas alternatifs :** Adoption du schéma CKKS (Arithmétique approximative) pour l'IA générative et le **Machine Learning** avancé, environnements nécessitant la manipulation de nombres à virgule flottante.

Merci de votre attention.

Questions ?